

CSE 753 Advanced Reinforcement Learning

Study Notes: Imitation Learning and Offline RL

(Lecture 08, page by page)

How to Use These Notes

These notes go through Lecture 08 (*Imitation Learning and Offline RL*) slide by slide, in the order the slides appear. For every idea and every equation you will find: what it is, what it means in plain English (aimed at someone with no background beyond basic algebra), a table explaining every symbol, an explanation of what the equation is saying, and a discussion of why the idea exists, what it can and cannot do, when to use it, and what happens if a design choice were made differently. A few places in the original slides contain small errors or use non-standard notation; these are flagged clearly with a “**Note on the slide**” so you are not confused by them on exam day.

1 Slide 2: What Is Imitation Learning?

What is the concept? Imitation Learning (IL) is a way of teaching a computer program (a *policy*) to perform a task by showing it recordings of an expert doing that task well — instead of telling the program a reward or a rule for what “good” means.

Plain-English explanation. Imagine you want to teach a robot to drive. You do not write down a mathematical rule for “good driving.” Instead, you record hours of a human driving: what the car’s sensors saw (the *state*) and what the driver did with the steering wheel and pedals (the *action*) at every instant. You then ask a computer to look at all of those recordings and learn a rule that maps “what the car sees” to “what to do,” by copying what the human did.

The equation.

$$\mathcal{D} = \{(s_1, a_1, \dots, s_T)\}$$

Symbol	What it means
s_t	The state (what the world / sensors look like) at time step t .
a_t	The action the expert took at time step t .
T	The number of time steps in one recorded trajectory (one “trip”).
$\mathcal{D}$	The whole dataset: the full collection of recorded trajectories.
π_{expert}	The unknown rule the expert was actually using to choose actions.
π_{θ}	The policy we want to train, with adjustable numbers (parameters) θ .

What the equation is saying. $\mathcal{D}$ is simply a list of recorded trips. Each trip is a sequence of “what did the world look like, what did the expert do” pairs. We never get to see the expert’s internal rule π_{expert} directly — only the footprints it left behind in the data.

Aspects of this concept.

- **Problem it solves:** it lets us build a working policy for tasks where writing down a reward function is hard (e.g., what number captures “good driving?”), as long as we can get someone to demonstrate the task.
- **Problem it cannot solve:** it only tells us what action the expert chose, never how good that action was compared to alternatives; a policy trained this way can never (in principle) beat the expert it copied.
- **Problems it gives rise to:** if we are not careful about the learning method, we run into the “mean-collapse” problem (Slide 4) and the “compounding error” problem (Slide 12) — both covered later.
- **When to use it:** when demonstrations are available or cheap to collect, and a good reward function is hard to write down.
- **When not to use it:** when there is no expert to copy, or when the goal is to do *better* than any available expert — that calls for reinforcement learning (RL) instead, which is what the rest of this lecture builds toward.
- **What if this were done differently?** If, instead of only (s, a) pairs, the dataset also included a reward r at every step, we could use offline RL (Slide 15 onward) and potentially outperform the expert, rather than just copy it.

2 Slide 3: Imitation Learning — Version 0 (Behavior Cloning, Deterministic)

What is the concept? The simplest possible way to do imitation learning: treat it as an ordinary supervised regression problem. For every recorded state, make the policy’s predicted action match the expert’s recorded action as closely as possible. This is called **Behavior Cloning (BC)**.

Plain-English explanation. Think of it like a fill-in-the-blank exercise: you show the network a state, it guesses an action, and you tell it exactly how wrong the guess was compared to what the expert actually did. You repeat this over the whole dataset, nudging the network’s numbers a little bit each time so its guesses get closer to the expert’s actions.

The equation.

$$\min_{\theta} \frac{1}{|\mathcal{D}|} \sum_{(s,a) \in \mathcal{D}} \|a - \hat{a}\|^2, \quad \hat{a} = \pi_{\theta}(s)$$

Symbol	What it means
θ	The adjustable numbers (parameters) inside the policy network.
$ \mathcal{D} $	The total number of (s, a) pairs in the dataset.
a	The action the expert actually took in state s .
$\hat{a}$	The action our policy predicts for that same state s .
$\pi_\theta(s)$	The policy: a function that takes one state in and outputs one action.
$\ \cdot\ ^2$	Squared distance between two action vectors (how far apart they are).
$\min_\theta$	We search for the parameters θ that make the total error smallest.

What the equation is saying. For every recorded state, measure how far the policy’s guess is from what the expert did, square that distance (so being very wrong is punished a lot more than being a little wrong), and average this over the whole dataset. Then adjust θ to make this average as small as possible.

Aspects of this concept.

- **Advantage:** extremely simple — it is exactly ordinary supervised regression, so all the standard machine-learning tools apply directly; fast and easy to train.
- **Disadvantage:** it assumes there is basically one “correct” action per state. It cannot represent a situation where two very different actions are both perfectly valid.
- **Problem it solves:** turns policy learning into a completely standard, well-understood regression problem — no reward, no exploration, no RL loop needed.
- **Problem it cannot solve:** if the expert data is *multimodal* (the expert sometimes did one thing and sometimes a very different thing in a similar situation), this method collapses both choices into their average, which can be a useless or dangerous action (see Slide 4).
- **What problem it gives rise to:** the mean-collapse failure, which motivates version 1 (Slide 6) with more expressive policies.
- **When to use it:** when the expert’s behavior is essentially *unimodal* — one sensible action per state, with only small variation.
- **When not to use it:** when the expert’s behavior is multimodal, i.e. there are multiple, very different, equally good actions in the same state.
- **What if this were done differently?** If we swapped the squared-error loss for a maximum-likelihood loss with an expressive distribution, we would get version 1 (Slide 6), which is able to represent more than one good action per state.

3 Slide 4: What Can Go Wrong? The Mean-Collapse Problem

What is the concept? This slide names and explains the specific failure of version-0 behavior cloning: when the expert’s actions are multimodal (more than one distinct, valid strategy), squared-error regression learns the *average* of those strategies, which is often an action that is unsafe and was never actually taken by anyone.

Plain-English explanation. Picture a highway scene: some human drivers, seeing a slow truck ahead, merge left; other drivers stay in their lane. Both are perfectly good choices. If you train a squared-error policy on this mixed data, it will not learn “sometimes merge, sometimes stay.” It will learn one single number that sits roughly *between* those two choices — which, on a real road, means steering the car into the gap between the lanes, straight toward the truck. That compromise action is worse than either of the two things any actual human did.

Mechanism (why this happens). For a fixed input, squared-error loss is minimized exactly when the prediction equals the *mean* of the target values for that input. This is a basic fact about squared error, not something specific to neural networks. If the true action distribution $p(a | s)$ has two separate “bumps” (modes), the mean falls in the valley between them — a point with almost no real probability under the data.

Worked numerical example. Suppose 75% of recorded examples for a given state are “stay straight” ($a \approx 0^\circ$) and 25% are “merge left” ($a \approx -2^\circ$ steering angle). The learned (mean) action is

$$\mathbb{E}[a | s] = 0.75 \times 0 + 0.25 \times (-2) = -0.5^\circ,$$

which matches the dashed line on the slide. This -0.5° action was never actually demonstrated by anyone; it straddles both lanes.

Aspects of this concept.

- **Name of the failure:** mean-seeking behavior (also called mode-averaging or mean-collapse).
- **Why it is dangerous, specifically:** the learned action is not a safe compromise between two valid strategies — it is a third action that lies in the gap between them and that no expert ever took. On a highway, it drives into the obstacle; in a surgical setting, it could mean cutting exactly where neither valid surgical approach cuts.
- **Problem it gives rise to:** the need for expressive (multimodal) policy classes, covered starting at Slide 5 and built in Slides 6–11.
- **What if this were done differently?** If the policy’s output distribution could represent two separate bumps instead of one number, it could put its probability mass on both “merge” and “stay,” rather than averaging them. This is exactly what Slides 5–11 build.

4 Slide 5: Neural Networks as Distribution Parameterizations

What is the concept? A neural network only ever outputs numbers; how those numbers are turned into an action depends on what kind of probability distribution you decided to wrap around the output. The expressive power of the *network* (how complicated a function it can compute) is

a completely different question from the expressive power of the *distribution family* you chose for its output.

Plain-English explanation. For a video-game character choosing among a small, fixed list of buttons (up, down, A, B), the network can output one probability per button. This is called a *categorical* distribution, and it can represent *any* pattern of preferences over those buttons — including “sometimes A, sometimes B, rarely up or down.” That is maximally expressive. But for a continuous action like a steering angle, the slide’s simple design has the network output just two numbers, μ (the center) and σ (the spread), of a single bell curve (Gaussian). A single bell curve can only ever have *one* peak. No matter how large or clever the network is, if you force its output through a single Gaussian, it can never represent “sometimes steer left, sometimes stay straight” as two separate peaks — it will always squash them into one peak in the middle, which is exactly the mean-collapse problem from Slide 4.

Aspects of this concept.

- **Advantage of the categorical (discrete) case:** naturally and fully multimodal — no extra machinery needed.
- **Disadvantage of the single-Gaussian (continuous) case:** unimodal by construction, regardless of how big or well-trained the underlying network is.
- **Problem it explains:** *why* Slide 3/4’s version-0 policy fails for continuous multimodal experts — it is not a training failure, it is a representational limitation baked into the choice of a single Gaussian.
- **What problem it gives rise to:** the need for continuous-action distribution families that *can* have more than one peak — Gaussian Mixture Models, autoregressive models, and diffusion/flow policies (Slides 7–11).
- **When to use a single Gaussian:** only when you are confident the expert behavior really is unimodal for continuous actions.
- **What if this were done differently?** If the continuous-action output were instead a mixture of several Gaussians (Slide 7), the same network architecture idea (numbers in, numbers out) becomes just as expressive as the categorical case.

5 Slide 6: Imitation Learning — Version 1 (Expressive Policies)

What is the concept? Instead of regressing to a single number (version 0), train the policy to output a full probability distribution over actions, and make the expert’s actual action as *likely as possible* under that distribution. This is **maximum likelihood estimation (MLE)**, and it opens the door to multimodal policies.

Plain-English explanation. Rather than saying “get as close as possible to this exact number,” we now say “make this exact action look as probable as possible under whatever shape of distribution you are outputting.” If the distribution family is expressive enough (can have more than one peak), the network can learn to put a peak over *each* of the expert’s distinct strategies, instead of averaging them together.

The equation.

$$\min_{\theta} -\mathbb{E}_{(s,a) \in \mathcal{D}} [\log \pi_{\theta}(a \mid s)]$$

Symbol	What it means
θ	Adjustable parameters of the policy network.
$\mathbb{E}_{(s,a) \in \mathcal{D}}$	Average over every (s, a) pair in the dataset.
$\pi_{\theta}(a \mid s)$	The probability (or probability density) the policy assigns to action a , given state s ; can be any shape, not just a single bell curve.
$\log \pi_{\theta}(a \mid s)$	The log-probability of the expert's actual action; higher means the policy thinks that action was more likely.
$-\mathbb{E}[\dots]$	We minimize the <i>negative</i> average log-probability, which is the same as maximizing the average log-probability.

What the equation is saying. Look at every expert action in the dataset. Push the policy's parameters so that the policy assigns *high probability* to exactly the actions the expert took, on average across the whole dataset.

Aspects of this concept.

- **Advantage:** more general than version 0; if paired with an expressive distribution family, it can represent multiple distinct valid strategies instead of averaging them.
- **Disadvantage:** by itself, this objective does not automatically fix mean-collapse — everything depends on how expressive $\pi_{\theta}(\cdot \mid s)$ is. If π_{θ} is still a single Gaussian, maximizing likelihood gives essentially the same mean-collapse result as version 0.
- **Problem it solves:** provides the general recipe (maximum likelihood) that Slides 7–11 plug different expressive distribution families into.
- **Problem it does not solve:** it does not address compounding error / distribution shift (Slide 12), which is a separate issue about *which states* the policy will see, not about how expressive its action distribution is.
- **When to use it:** whenever the expert's behavior might be multimodal, i.e. almost always for real-world tasks with more than one reasonable strategy.
- **What if this were done differently?** If we kept this exact objective but plugged in a single Gaussian for π_{θ} , we would be back to (a probabilistic version of) version 0 — proof that the objective alone is not what fixes mean-collapse; the distribution family matters just as much.

6 Slide 7: Gaussian Mixture Models (GMMs)

What is the concept? A Gaussian Mixture Model represents the action distribution as a weighted combination of several separate bell curves (Gaussians), each one representing a different possible “strategy.”

Plain-English explanation. Imagine several bell-shaped hills, each centered on a different action (a different strategy the expert might use), each with its own width and its own “how common is this strategy” weight. Add all these hills together (after scaling each by its weight) and you get one bumpy probability curve that can have several separate peaks — one per strategy — instead of being forced into a single hill.

The equations.

$$\{w_k(s), \mu_k(s), \Sigma_k(s)\}_{k=1}^K \quad (1) \quad p(a_i | s_i) = \sum_{k=1}^K w_k(s_i) \mathcal{N}(a_i | \mu_k(s_i), \Sigma_k(s_i)) \quad (2)$$

$$\max_{\theta} \sum_{i=1}^N \log p(a_i | s_i) \quad (3) \quad \mathcal{L}(\theta) = - \sum_{i=1}^N \log \left[\sum_{k=1}^K w_k(s_i) \mathcal{N}(a_i | \mu_k(s_i), \Sigma_k(s_i)) \right] \quad (4)$$

Symbol	What it means
K	The number of Gaussian “strategies” (mixture components) the network can use; chosen ahead of time.
$w_k(s)$	How likely strategy k is, given state s (all w_k for a fixed s add up to 1).
$\mu_k(s)$	The center (typical action) of strategy k , given state s .
$\Sigma_k(s)$	The spread/uncertainty (covariance) of strategy k , given state s .
$\mathcal{N}(a \mu, \Sigma)$	The height of a Gaussian bell curve with center μ and spread Σ , evaluated at action a .
$p(a_i s_i)$	The overall probability density of action a_i : the weighted sum of all K strategies’ bell curves.
N	The number of (s, a) training examples.
$\mathcal{L}(\theta)$	The negative log-likelihood loss actually minimized during training (equivalent to maximizing eq. (3)).

What the equations are saying. Equation (1): for every state, the network outputs K separate “strategies,” each with its own weight, center, and spread. Equation (2): the overall probability of an action is a weighted blend of all K bell curves. Equations (3)–(4): train the network so that, averaged over the dataset, the expert’s actual actions get high probability under this blended, multi-peaked curve.

Worked numerical example (checking the notation). Suppose two strategies have means -30° and $+30^\circ$, each with standard deviation $\sigma = 2$ (so $\Sigma = \sigma^2 = 4$, since the second number inside $\mathcal{N}(\cdot; \text{mean}, \text{variance})$ is the *variance*, not the standard deviation). If we ask for the density at $a = 0^\circ$ (right in between the two peaks), each Gaussian’s density is essentially zero there, because 0° is 15 standard deviations away from either mean ($30/2 = 15$). So $p(a = 0 | s) \approx 0.000$ — $a = 0$ sits in the trough between the modes, exactly the unsafe “average” action that Slide 4 warned about. A GMM correctly assigns it almost no probability, whereas a single Gaussian (Slide 5) would have centered its one peak right there.

Aspects of this concept.

- **Advantage:** can represent several genuinely distinct action strategies at once; the density has a clean closed-form formula, is easy to sample from, and is easy to differentiate for training.
- **Disadvantage:** the number of strategies K must be fixed in advance (cannot grow on its own), and it scales poorly to actions with many dimensions (the covariance matrices get large and hard to estimate).
- **Problem it solves:** directly fixes the mean-collapse problem from Slide 4 by letting the output have multiple peaks.
- **Problem it cannot solve:** it does not, by itself, model rich dependencies between different dimensions of a single action vector unless the covariance Σ_k is allowed to be a full (not just diagonal) matrix, which is expensive; it also does not address compounding error.
- **When to use it:** when you have a good guess of how many distinct strategies exist (a small, known K) and actions are low-to-moderate dimensional.
- **When not to use it:** for very high-dimensional, richly structured actions where the individual dimensions strongly depend on one another — an autoregressive model (Slide 8) is often a better fit there.
- **What if this were done differently?** If $K = 1$, a GMM is exactly a single Gaussian — back to the non-expressive case of Slide 5. If K is set far larger than the number of real strategies, the model can overfit or become unstable to train.

7 Slide 8: Multimodal Actions — The Chain-Rule (Autoregressive) Idea

What is the concept? Instead of representing a multi-dimensional action all at once (like a GMM does), break the action into its individual dimensions and predict them one at a time, each new dimension conditioned on all the dimensions already chosen. This uses the **chain rule of probability**, which is always mathematically exact.

Plain-English explanation. Suppose an action has several parts (e.g., steering angle *and* speed). Instead of guessing both at once, first guess the steering angle. Then, *knowing* what steering angle was just chosen, guess the speed (a sharp turn probably means a slower speed, for example). This lets the model capture how the different parts of an action relate to each other, which a single joint distribution might miss if it treats the dimensions independently.

The equations.

$$a_i = (a_{i,1}, a_{i,2}, \dots, a_{i,D}) \quad (5) \qquad p_\theta(a_i \mid s_i) = \prod_{d=1}^D p_\theta(a_{i,d} \mid s_i, a_{i,1:d-1}) \quad (6)$$

$$\max_{\theta} \sum_{i=1}^N \log p_\theta(a_i \mid s_i) \quad (7) \qquad \mathcal{L}(\theta) = - \sum_{i=1}^N \sum_{d=1}^D \log p_\theta(a_{i,d} \mid s_i, a_{i,1:d-1}) \quad (8)$$

Symbol	What it means
D	The number of separate dimensions (parts) in one action vector.
$a_{i,d}$	The d -th dimension of the i -th training action.
$a_{i,1:d-1}$	All the dimensions of action i chosen <i>before</i> dimension d .
$p_{\theta}(a_{i,d} \mid s_i, a_{i,1:d-1})$	The probability of dimension d , given the state and every dimension already decided.
$\prod_{d=1}^D$	Multiply the per-dimension probabilities together (chain rule).

What the equations are saying. Equation (5) just says an action can have several parts. Equation (6) says the probability of the whole action equals the probability of the first part, times the probability of the second part given the first, times the probability of the third part given the first two, and so on — this is always exactly true for any joint distribution, by the definition of conditional probability. Equations (7)–(8) say: train by maximizing (equivalently, minimizing the negative of) the total log-probability of the expert’s actual action, summed over every dimension and every training example.

Aspects of this concept.

- **Advantage:** the factorization in equation (6) is *exact* — no information is lost — and it explicitly captures dependencies between different dimensions of the same action, which a simple GMM with independent dimensions cannot.
- **Disadvantage:** actions must be generated one dimension at a time in sequence, which is slower at execution time than predicting everything in one shot; an error in an early dimension can affect every later dimension of the *same* action.
- **Problem it solves:** models rich correlations *within* a single action (e.g., steering and speed changing together), not just multiple separate strategies.
- **When to use it:** high-dimensional, structured actions where the dimensions clearly interact (e.g., the joint angles of a robot arm).
- **What if this were done differently?** If each dimension were instead predicted independently of the others (no conditioning on $a_{i,1:d-1}$), the model could not represent any relationship between dimensions — e.g., it could never learn that “sharp turn implies slower speed.”

8 Slide 9: Autoregressive Models — Generating the Action

What is the concept? This slide shows how an autoregressive policy actually *produces* an action once trained: dimension by dimension, sampling each one from its own conditional distribution.

Plain-English explanation. Once trained, using the policy is like a chain of small decisions: pick dimension 1; now that dimension 1 is fixed, pick dimension 2 knowing dimension 1; keep going until every dimension of the action has been chosen.

The equations.

$$\pi_{\theta}(a \mid s) = \prod_{d=1}^D \pi_{\theta}(a_d \mid s, a_{<d}) \quad (9) \qquad s \rightarrow a_1 \rightarrow a_2 \rightarrow \cdots \rightarrow a_D \quad (10)$$

Symbol	What it means
$a_{<d}$	Shorthand for $(a_1, \dots, a_{d-1})$: everything chosen before dimension d .
$a_1 \sim \pi_{\theta}(a_1 \mid s)$	Sample the first dimension from its distribution, given only the state.
$a_2 \sim \pi_{\theta}(a_2 \mid s, a_1)$	Sample the second dimension, now also conditioned on the first dimension just chosen.
$s \rightarrow a_1 \rightarrow \cdots \rightarrow a_D$	The generation order: state first, then each action dimension in turn.

What the equations are saying. Equation (9) is identical in spirit to equation (6) on the previous slide — it is the same chain-rule factorization, just written for execution rather than training. Equation (10) shows this as a left-to-right generation chain: state in, then one dimension of the action out at a time, each new dimension informed by all the previous ones.

Aspects of this concept. This slide is a direct continuation of Slide 8, so the advantages, disadvantages, and use cases are the same. The one new point worth remembering: this factorization explicitly models dependencies between action dimensions, and the **Decision Transformer** (next slide) is a concrete, well-known architecture built on exactly this autoregressive idea.

9 Slide 10: Decision Transformer

What is the concept? A concrete architecture that turns imitation learning into a sequence-modeling problem (the same kind of problem language models solve), by feeding a Transformer a sequence of (desired future return, state, action) triples and asking it to predict the next action.

Plain-English explanation. Instead of thinking in terms of states and rewards the usual RL way, treat the whole problem like predicting the next word in a sentence. The “sentence” here is a sequence of tokens: “how much reward do I still want to get,” “what do I currently see,” “what did I just do,” repeated over time. A *causal* Transformer (one that can only look at the past, never the future) reads this sequence and predicts the next action.

Symbol	What it means
$\hat{R}_t$	The return-to-go: how much total reward we would like to still receive from this point onward.
s_t	The state at time t .
a_t	The action at time t .
Causal transformer	A Transformer that, when predicting position t , is only allowed to look at positions $1, \dots, t$, never anything later.
Linear decoder	The final small layer that turns the Transformer’s internal numbers into an actual action.

Aspects of this concept.

- **Advantage:** reuses powerful, well-understood sequence models (Transformers); at test time, you can ask for a high desired return and the model will try to act like a higher-performing agent, entirely through ordinary supervised training (no bootstrapped value function needed).
- **Disadvantage:** needs a reasonably long context window and has the usual quadratic-in-length cost of attention; because it only imitates sequences that were actually observed, asking for an unrealistically high return can make it behave poorly (it has nothing real to imitate at that return level).
- **Problem it solves:** avoids the instability that comes from bootstrapped value learning (TD updates), replacing it with plain supervised sequence prediction.
- **Problem it does not solve:** because it does not perform explicit value backups, it generally cannot *stitch* together good pieces of different trajectories into a new, better trajectory the way TD-based offline RL can (see Slide 16).
- **When to use it:** when you have a large offline dataset of trajectories with a range of returns and want a simple, stable, supervised training recipe.
- **What if this were done differently?** If the Transformer were *non-causal* (allowed to look at future tokens), it would be cheating by peeking at information it will not have during real-time control — the causal mask is essential for the model to be usable at actual decision time.

10 Slide 11: Diffusion Policy

What is the concept? A way to turn random noise into a realistic action by learning a “velocity field” that gradually pushes noisy points toward the shape of the real action distribution.

Note on the slide. This slide is titled “Diffusion Policy,” but the actual procedure shown — a straight linear interpolation between noise and data, and a single learned velocity field integrated with simple Euler steps — is what is technically called **flow matching** (or rectified flow), not classic diffusion (DDPM). Classic diffusion gradually corrupts data with many small noise steps and learns to reverse that corruption stochastically. Flow matching, shown here, learns one direct “straight-line” velocity field instead. For the exam, answer using exactly the procedure written on the slide; if you want to show extra precision, you can note in one sentence that the mechanism is really a learned vector field (flow matching), not iterative denoising.

Plain-English explanation. Start with a completely random point (pure noise). Learn a “which direction should I walk” rule (the velocity field) that, if you take many small steps following it, will carry that random point all the way to a realistic action. Different random starting points end up at different final actions — so this approach is naturally multimodal: different noise samples can land near different expert strategies.

The steps (training and inference).

Symbol	What it means
$x_1 \sim \mathcal{D}$	A real action sampled from the dataset.
$x_0 \sim \mathcal{N}(0, 1)$	A random noise sample.
$t \sim p(t)$	A random “progress fraction” between 0 (all noise) and 1 (all data).
$x_t = tx_1 + (1-t)x_0$	A blend of noise and data: the point that is t fraction of the way from noise to data.
$v_\theta(x_t, t)$	The network’s prediction of “which direction to move” at point x_t and progress t (conditioned on the state s_t for imitation learning).
δ	The step size used when numerically integrating the velocity field.

Training: sample $x_1 \sim \mathcal{D}$ and $x_0 \sim \mathcal{N}(0, 1)$; sample t ; interpolate $x_t = tx_1 + (1-t)x_0$; minimize $\|v_\theta(x_t, t) - (x_1 - x_0)\|^2$ (i.e., teach the network to predict the straight-line direction from noise to data).

Inference: start from noise $x_0 \sim \mathcal{N}(0, 1)$; repeatedly step $x_{t+\delta} = x_t + v_\theta(x_t, t)\delta$ for $t = 0, \delta, 2\delta, \dots, 1 - \delta$; return x_1 as the final action.

Aspects of this concept.

- **Advantage:** highly expressive — can represent very complex, high-dimensional, arbitrarily-shaped multimodal action distributions, more flexible than a fixed- K GMM.
- **Disadvantage:** generating one action requires several iterative steps (slower at test time than a single forward pass through a GMM or autoregressive model); more moving parts to implement and tune (step size, number of steps).
- **Problem it solves:** represents very rich, complex continuous multimodal distributions that a fixed-size GMM or a simple autoregressive model may struggle to capture.
- **Problem it does not solve:** like every method in Slides 6–11, it only improves *how expressive* the action distribution is; it does nothing about compounding error / distribution shift (Slide 12), which is a separate, data-coverage problem.
- **When to use it:** complex, high-dimensional continuous control (e.g., robot manipulation) where test-time compute for several iterative steps is affordable.
- **When not to use it:** latency-critical, real-time control where only a single fast forward pass is acceptable.
- **What if this were done differently?** Using a larger step size δ (fewer integration steps) makes generation faster but less accurate (bigger discretization error) — a direct speed-versus-accuracy trade-off.

11 Slide 12: Problems with Imitation Learning — Compounding Error

What is the concept? Even with a perfectly expressive policy (Slides 6–11), imitation learning has a second, completely different failure mode: small mistakes can snowball because the policy’s

own actions change what state it sees next.

Plain-English explanation. In ordinary supervised learning (like recognizing pictures of cats), each example is independent — one wrong guess does not change which picture you see next. But in imitation learning, the policy’s action actually changes the world: a slightly wrong steering command puts the car in a slightly different position than any human driver was ever recorded in. The policy was never trained on that slightly-off position, so it is more likely to make another, possibly bigger, mistake there — pushing the car even further from anywhere in the training data. This keeps happening, and small errors compound into large ones.

Mechanism, step by step (this is the full failure chain).

1. The predicted action affects the next state; states are not independent of the policy, unlike ordinary supervised learning.
2. A small prediction error nudges the agent slightly off the states the expert ever demonstrated.
3. The policy was never trained on this new, off-distribution state, so its prediction there is unreliable and tends to be more wrong.
4. This repeats at every subsequent step, and errors compound over the length of the trajectory — growing much faster than they would if each mistake were independent.

Aspects of this concept.

- **Name of the failure:** compounding error, also called distribution shift.
- **Two remedies shown on the slide:**
 1. *Collect a lot of demonstration data and hope for the best.* Advantage: requires no algorithm change, very simple. Disadvantage: does not really fix the underlying issue — it only reduces the *chance* of drifting off-distribution; you can never demonstrate every possible state in advance, especially in a large or continuous state space.
 2. *Collect corrective behavior data* — this leads directly to DAgger, the next slide.
- **Problem it gives rise to:** the DAgger algorithm (Slide 13), and, more broadly, the entire motivation for offline RL, which uses reward information to be more robust to imperfect data coverage.

12 Slide 13: DAgger — Dataset Aggregation

What is the concept? DAgger directly fixes compounding error by changing *where* the training data comes from: instead of only collecting states the expert visited, it collects states the *learner itself* visits, and labels those states with what the expert would have done there.

Plain-English explanation. Run your current (still imperfect) policy in the real environment and see exactly where it goes. At each of those states, go ask the real expert, “what would you have done here?” Add these new (state, expert-action) pairs to your dataset, and retrain on everything collected so far. Repeat this loop. Over time, the training data starts to match exactly the states the policy will actually encounter, so there is no more gap between training and deployment.

The equation.

$$\mathcal{D} \leftarrow \mathcal{D} \cup \{(s', a^*)\}, \quad s' \sim \pi_\theta, \quad a^* \sim \pi_{\text{expert}}(\cdot \mid s')$$

Symbol	What it means
$s' \sim \pi_\theta$	A state actually visited by rolling out the current learned policy.
$a^* \sim \pi_{\text{expert}}(\cdot \mid s')$	The action the real expert would take at that visited state, queried on demand.
$\mathcal{D} \leftarrow \mathcal{D} \cup \{(s', a^*)\}$	Add this new labeled pair to the growing dataset (“aggregate”).

Full loop. Roll out π_θ to get visited states $\rightarrow$ query the expert at those states $\rightarrow$ aggregate the new pairs into $\mathcal{D} \rightarrow$ retrain $\min_\theta \mathcal{L}(\pi_\theta, \mathcal{D}) \rightarrow$ repeat.

Aspects of this concept.

- **Advantage:** directly closes the mismatch between training data and deployment data that caused compounding error — the policy is trained exactly on the states it will actually visit.
- **Disadvantage:** requires an *interactive* expert available throughout training, one that can be asked to label any state on demand. This is often infeasible — a surgeon cannot realistically sit and label every single state a surgical robot visits during training. The rollout-query-aggregate-retrain loop is also slow and expensive to run repeatedly.
- **Problem it solves:** eliminates distribution-shift-driven compounding error at its root.
- **Problem it cannot solve:** it does not help when no interactive expert is available, or when the “expert” has no good answer for a truly bizarre state the learner has drifted into.
- **When to use it:** when a cheap, available interactive expert exists (e.g., a scripted expert in simulation, or a supervisor watching a live feed).
- **When not to use it:** when only a fixed batch of past demonstrations exists and no one can be asked new questions (e.g., historical human driving logs) — offline methods (Slides 14 onward) are the fallback there.
- **What if this were done differently?** If step 2 used the *old* dataset’s nearest recorded action instead of asking the true expert fresh, the new label would not actually reflect what should be done at that specific new state — the whole point of DAgger is that the correction has to come from the real expert reacting live to the state actually visited.

13 Slide 14: Why Offline RL?

What is the concept? This slide contrasts the usual online RL loop (collect data, update policy, repeat) with offline RL, where the policy is trained once on a fixed, already-collected dataset and never interacts with the real environment during training.

Plain-English explanation. Online RL is like learning to ride a bike by actually riding it, falling, and adjusting — over and over, in real time. Offline RL is like being handed a giant pile of videos of other people riding bikes (some skilled, some not) and being asked to learn to ride well just by studying those videos, without ever getting on a bike yourself during training.

Online RL	Offline RL
Collect data, then update the policy on the latest data (or all data so far), and repeat.	Given one static, already-collected dataset; train the policy purely on that provided dataset.

Aspects of this concept.

- **Advantages of offline RL:** it can reuse data other people or systems already collected; live data collection may be risky or unsafe (a self-driving car or a surgical robot exploring randomly in the real world is dangerous); data can be reused across different experiments, projects, robots, or institutions; and offline pretraining can be blended with a later stage of online fine-tuning.
- **Disadvantage:** the fixed dataset may not cover every state or action well, and it may not contain optimal behavior — this data-coverage gap is the seed of the out-of-distribution (OOD) action problem discussed in Slides 17–18.
- **Problem it solves:** lets us learn from safe, already-existing data instead of expensive or dangerous live trials.
- **Problem it does not solve:** it does not guarantee the data covers everything the learned policy might need — care is required so the policy does not over-trust regions the data barely covers.
- **When to use it:** safety-critical or expensive-to-explore domains where logged data already exists.
- **When not to use it:** domains where cheap, safe live exploration is available — online RL may simply be easier there.

14 Slide 15: Offline RL — Formal Setup

What is the concept? This slide gives the precise mathematical definition of the offline RL problem: a dataset generated by some unknown “behavior policy,” and a goal that is still the usual RL goal — maximize expected total reward under the policy we are training, not under the policy that generated the data.

Plain-English explanation. We are handed a big table of (state, action, next state, reward) records, produced by *some* policy π_β that we do not control and may not even fully know (it could even be a mix of several different people’s or systems’ behavior). Even though we only get to look at π_β ’s data, our actual goal has not changed: we still want to find the best possible policy π_θ to run in the real environment.

The equations.

$$p_{\pi_\beta}(\tau) = p(s_1) \prod_{t=1}^T \pi_\beta(a_t \mid s_t) p(s_{t+1} \mid s_t, a_t) \quad \max_{\theta} \mathbb{E}_{p_{\pi_\theta}} \left[\sum_t r(s_t, a_t) \right]$$

Symbol	What it means
$\mathcal{D} : \{(s, a, s', r)\}$	The offline dataset: recorded (state, action, next state, reward) tuples.
π_β	The unknown “behavior policy” that actually generated the dataset (possibly a mixture of several policies).
$p(s_1)$	The probability distribution over starting states.
$p(s_{t+1} \mid s_t, a_t)$	The environment’s own dynamics: how the world responds to an action (not something we control or choose).
p_{π_θ}	The distribution over trajectories that would occur if we actually ran our trained policy π_θ in the environment.
$r(s_t, a_t)$	The reward received at each step.

What the equations are saying. The first equation just says: a recorded trajectory’s probability is built from the starting-state distribution, the behavior policy’s action choices, and the environment’s own dynamics — multiplied together over every time step. The second equation is the important one: our objective is expected total reward under π_θ (the policy we are training), *not* under π_β (the policy that collected the data). This mismatch between “what generated the data” and “what we’re evaluating” is the whole source of difficulty in offline RL.

Aspects of this concept.

- **What problem it defines:** formalizes offline RL’s goal precisely so we can reason about whether an algorithm is even trying to solve the right problem.
- **Key subtlety:** because the objective’s expectation is under π_θ while the only data we have came from π_β , any algorithm that is not careful can end up trusting parts of π_θ ’s behavior that were never actually observed.
- **What if this were done differently?** If we instead maximized the expected reward under p_{π_β} (the data’s own distribution), we would simply be *evaluating* the behavior policy that already exists, not improving on it. The entire value of offline RL comes from doing better than π_β while only ever looking at π_β ’s data.

15 Slide 16: Offline RL versus Imitation Learning — Stitching

What is the concept? This slide makes the central case for offline RL over imitation learning: a good offline RL algorithm can combine (“stitch together”) good pieces from *different* trajectories into a single new trajectory that is better than any trajectory actually demonstrated. Plain imitation learning can never do this.

Plain-English explanation. Imagine two separate recorded trips through a maze of connected states. One trip successfully reaches the goal (s_9) after passing through s_7 ; a different, separate trip passes through $s_3 \rightarrow s_7$ but then goes somewhere unhelpful and ends badly. Even though no single recorded trip goes cleanly from the start all the way to the goal via $s_3 \rightarrow s_7 \rightarrow s_9$, a good offline RL algorithm can realize that “ s_7 leads somewhere great” (learned from the first trip) and connect that fact to “ $s_3 \rightarrow s_7$ is a transition that exists in the data” (from the second trip) — gluing two half-good trips into one fully good path.

The key claim on the slide. *Imitation methods cannot outperform the policy that collected the data. Offline RL can leverage reward information to outperform the behavior policy. Good offline RL methods can stitch together good behaviors.*

Mechanism of stitching (why it works, step by step).

1. From the trajectory that reaches the goal via $s_7 \rightarrow s_9$ (with reward $r(s_9) = +1$), temporal-difference (TD) learning assigns a high learned value to s_7 , because that value backs up from the good reward at s_9 .
2. A *different* trajectory contains the transition $s_3 \rightarrow s_7$. Its TD target for that step is $r + \gamma V(s_7)$ — it uses the *learned* value of s_7 , not whatever return that second trajectory itself happened to observe afterward.
3. Because $V(s_7)$ is high (learned from the *first* trajectory), the transition $s_3 \rightarrow s_7$ gets credited with a high value, even though the trajectory it was recorded in did not end well.
4. This is only possible because of **bootstrapping** (TD learning): the value used in the target comes from the learned value function, not from what actually happened afterward in that particular trajectory.

Does it matter whether we use TD or Monte Carlo? Yes. Monte Carlo (MC) value estimation would instead credit the $s_3 \rightarrow s_7$ transition with the *actual observed return* of the trajectory it came from (which ended badly) — it has no mechanism for borrowing the good outcome discovered in a completely different trajectory. So stitching *depends specifically on using TD*, not MC.

Aspects of this concept.

- **Advantage of offline RL over plain imitation learning:** can exceed the performance of the data-collecting policy by recombining good sub-behaviors that were never demonstrated together in a single trajectory.
- **Why this is the deepest idea in this section (justify-the-design):** stitching is not a side benefit of TD learning — it is the *direct consequence* of using a learned, bootstrapped value target instead of the trajectory’s own observed return.
- **What if this were done differently?** If value estimation used Monte Carlo returns instead of TD bootstrapping, stitching would not happen — this is exactly why AWR (Slide 21), which uses Monte Carlo, cannot outperform the behavior policy the way AWAC or IQL (which use TD) potentially can.

16 Slides 17–18: Why Not Just Use Ordinary Off-Policy Algorithms?

What is the concept? These two slides show what happens if we take an ordinary off-policy actor-critic method (like SAC, designed for *online* RL with a replay buffer) and simply apply it to a fixed, static, offline dataset. The result is a specific, serious failure: the learned Q-function becomes badly overestimated on actions that were never in the data.

Plain-English explanation. The usual actor-critic recipe fits a Q-function (“how good is this action here?”) and then improves the policy to pick whatever action the Q-function currently says is best. When training online, if the Q-function is wrong about some action, the agent can simply *try it* and find out the truth, correcting the mistake. But offline, there is no real environment to check against. If the Q-function is (wrongly) very optimistic about some action that never appears in the dataset, the policy will happily chase that mistake, and nothing will ever correct it.

The equations.

$$\min_{\phi} \sum_{(s,a,s') \sim \mathcal{D}} \left\| \hat{Q}_{\phi}^{\pi_{\theta}}(s, a) - \left(r(s, a) + \gamma \mathbb{E}_{a' \sim \pi_{\theta}(\cdot | s')} [\hat{Q}_{\phi}^{\pi_{\theta}}(s', a')] \right) \right\|^2$$

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_i \nabla_{\theta} \log \pi_{\theta}(a_i^{\pi^*} | s_i) \hat{Q}_{\phi}^{\pi_{\theta}}(s_i, a_i^{\pi^*}), \quad a_i^{\pi^*} \sim \pi_{\theta}(a | s_i)$$

Symbol	What it means
$\hat{Q}_{\phi}^{\pi_{\theta}}(s, a)$	The learned critic: an estimate of how good action a is in state s , for the current policy π_{θ} .
$a' \sim \pi_{\theta}(\cdot s')$	A <i>freshly sampled</i> next action from the <i>current</i> policy — not an action read from the dataset.
$a_i^{\pi^*} \sim \pi_{\theta}(a s_i)$	A freshly sampled action from the current policy, used to improve the actor by climbing the critic.
Out-of-distribution (OOD) action	An action that the dataset never actually contains for a given state.
Data support	The range of actions the dataset actually contains for a given state.

What the equations are saying. The critic update tries to make $\hat{Q}$ consistent with the reward plus the value of whatever action the *current* policy would pick next (not the action actually recorded in the data). The actor update then pushes the policy toward whatever action currently maximizes $\hat{Q}$. Both steps repeatedly ask the critic about actions the current policy *chooses*, which may well not be actions ever seen in the dataset.

Mechanism of the failure (the OOD chain — memorize this in order).

1. The critic’s bootstrapped target requires $\mathbb{E}_{a' \sim \pi_{\theta}(\cdot | s')} [\hat{Q}(s', a')]$, and a' need not appear anywhere in $\mathcal{D}$.
2. $\hat{Q}$ is unreliable — essentially untrained noise — on such out-of-distribution (OOD) actions, since nothing in the data constrains its value there.

3. Because the actor update explicitly maximizes $\hat{Q}$, it actively seeks out exactly the actions where $\hat{Q}$ happens to be (wrongly) high.
4. Those inflated, incorrect values then feed into the *next* round's TD target, so Q-values across the board become substantially overestimated.
5. The learned policy drifts further and further from the real behavior policy, with no real environment interaction available to ever catch or correct the mistake. Online RL escapes this loop by actually trying the action and observing the true reward; offline, the loop stays closed forever. This is exactly why it is the *offline* setting specifically that breaks this algorithm.

Aspects of this concept.

- **Advantage of this recipe (in its native, online setting):** efficient reuse of a replay buffer while still being able to correct mistakes through live interaction.
- **Disadvantage (offline specifically):** catastrophic, self-reinforcing Q-value overestimation.
- **When not to use this recipe as written:** whenever training purely offline on a fixed, static dataset.
- **When it is fine to use:** online settings, where the agent can actually try the action it is curious about and observe the real reward.
- **What if this were done differently?** If a' were instead sampled from the dataset itself ($a' \sim \mathcal{D}$) rather than from π_θ , the OOD query disappears, but now the critic is evaluating something closer to Q^{π_β} (the old behavior policy) rather than the current policy π_θ — trading one problem (OOD overestimation) for another (evaluating the wrong policy). This exact trade-off reappears explicitly as a noted “alternative” in AWAC (Slide 23).

17 Slide 19: Offline RL — A Baseline: Filtered Behavior Cloning

What is the concept? A very simple way to use reward information without touching Q-functions or value functions at all: rank whole trajectories by their total reward, keep only the good ones, and run ordinary behavior cloning on what remains.

Plain-English explanation. Score every recorded trip by how much total reward it earned. Throw away the low-scoring trips. Pretend the remaining, better trips are the only dataset that ever existed, and run plain behavior cloning (Slide 3/6) on it.

The equations.

$$r(\tau) = \sum_{(s_t, a_t) \in \tau} r(s_t, a_t) \quad \tilde{\mathcal{D}} : \{\tau \mid r(\tau) > \eta\} \quad \max_{\theta} \sum_{(s, a) \in \tilde{\mathcal{D}}} \log \pi_{\theta}(a \mid s)$$

Symbol	What it means
$r(\tau)$	The total reward earned across an entire trajectory τ .
η	A threshold return; trajectories must beat this to be kept.
$\tilde{\mathcal{D}}$	The filtered dataset: only the trajectories with $r(\tau) > \eta$.
$\max_{\theta} \sum \log \pi_{\theta}(a \mid s)$	Ordinary maximum-likelihood behavior cloning, applied only to $\tilde{\mathcal{D}}$.

What the equations are saying. First, add up the reward across each whole trip. Second, keep only the trips whose total beats some cutoff η (e.g., the top $k\%$ of trips). Third, forget about rewards entirely and just imitate whatever is left, exactly like ordinary behavior cloning.

Aspects of this concept.

- **Advantage:** extremely simple; no value function, no Q-learning, and therefore no out-of-distribution query risk at all — it fully reuses the plain behavior-cloning machinery.
- **Disadvantage:** filtering happens at the level of the *whole trajectory*. If a trajectory clears the threshold, *every* (s, a) pair inside it survives into $\tilde{\mathcal{D}}$ — including any individual bad or mediocre actions that happened to occur inside an overall good trip. Also, if only one kept trajectory ever visits a particular state, the learned action there is entirely determined by that one trajectory, with no evidence that it was actually the best possible action at that state.
- **Problem it solves:** a low-risk way to make some use of reward information, without any of the machinery (or the OOD danger) of Q-learning.
- **Problem it cannot solve:** it cannot stitch trajectories together (Slide 16), and it cannot tell a genuinely good action apart from a merely mediocre action that happened to ride along inside an otherwise-good trajectory.
- **When to use it:** as a quick, low-risk baseline when return-labeled trajectories are available.
- **When not to use it:** when very few trajectories clear the threshold, or when good actions are scattered as isolated good moments inside otherwise mediocre trajectories (that signal is lost, since only whole trajectories are judged).
- **What if this were done differently?** If filtering were applied at the level of *individual* (s, a) pairs (e.g., by a per-step advantage estimate) instead of whole trajectories, we would move toward weighted imitation learning (Slide 20) rather than filtered behavior cloning.

18 Slide 20: Weighted Imitation Learning

What is the concept? Instead of a hard yes/no cutoff (filtered BC), give every individual (s, a) pair a *soft* weight based on how much better than average that specific action was — its **advantage**.

Plain-English explanation. Recall the advantage function: $A_\pi(s, a) = Q_\pi(s, a) - V_\pi(s)$, i.e., how much better action a is compared to the average action in state s . Weighted imitation learning uses this number to decide how hard to imitate each recorded action: actions with a big positive advantage get imitated strongly (a big weight); actions with an advantage near zero or negative barely influence training at all, but are not thrown away completely the way filtered BC would throw away an entire trajectory.

The equations.

$$A_\pi(s, a) = Q_\pi(s, a) - V_\pi(s) \qquad \theta \leftarrow \arg \max_{\theta} \mathbb{E}_{s, a \sim \mathcal{D}} [\log \pi_\theta(a \mid s) \exp(A(s, a))]$$

Symbol	What it means
$Q_\pi(s, a)$	Expected total future reward if you take action a in state s , then follow π afterward.
$V_\pi(s)$	Expected total future reward from state s , averaged over the actions π would itself choose.
$A_\pi(s, a)$	How much better (or worse) action a is than the average action in state s .
$\exp(A(s, a))$	The soft weight given to this (s, a) pair during imitation: always positive, growing quickly for large positive advantages.

What the equations are saying. The first equation defines “how good was this action, relative to average.” The second equation says: do ordinary maximum-likelihood imitation (as in Slide 6), but multiply each example’s contribution by $\exp(A(s, a))$ — so better-than-average actions get imitated much more strongly than merely average or below-average ones.

Aspects of this concept.

- **Advantage over filtered BC:** smoother and more fine-grained — it uses information from *every* sample rather than an all-or-nothing cutoff, and it discriminates at the level of individual (s, a) pairs rather than whole trajectories.
- **Disadvantage:** needs an advantage estimate $A(s, a)$, which itself must be learned from a V and/or Q function — introducing its own source of estimation error. (Slides 21–26 are all about *how* to estimate this advantage well.)
- **Problem it solves:** gives a smoother, more data-efficient way to bias imitation toward better actions than a hard trajectory-level filter.
- **Problem it does not solve by itself:** it does not specify how to estimate $A(s, a)$ — that choice (Monte Carlo vs. TD, on which policy) is exactly what distinguishes AWR, AWAC, and IQL from each other.
- **When to use it:** whenever a reasonable Q/V estimate is available and you want a policy-gradient-free way to bias imitation toward good behavior.
- **What if this were done differently?** If the weight were simply $A(s, a)$ instead of $\exp(A(s, a))$, negative advantages would produce *negative* weights, which is not a valid weighting for a log-likelihood (weights in weighted MLE must stay positive). The exponential also makes large advantages dominate multiplicatively, sharpening the imitation toward the very best actions.

19 Slide 21: Advantage-Weighted Regression (AWR)

What is the concept? A complete, two-step algorithm that fills in weighted imitation learning (Slide 20) with a concrete way to estimate the advantage: fit a value function using plain Monte Carlo regression, then run weighted imitation with a temperature knob α .

Plain-English explanation. Step 1: for every state, look at the total reward that was *actually observed* afterward in the data, and train $\hat{V}$ to regress toward that number directly — an ordinary supervised-learning target, no bootstrapping. Step 2: run weighted imitation exactly as in Slide 20, where the “how good was this action” score is (the observed future return) minus (this state’s estimated average value), divided by a knob α before being exponentiated, so the weights do not explode into astronomically large numbers.

The equations.

$$\min_{\phi} \sum_{s_t \sim \mathcal{D}} \left\| \hat{V}_{\phi}^{\pi_{\beta}}(s_t) - \sum_{t'=t}^T r(s_{t'}, a_{t'}) \right\|^2$$

$$\max_{\theta} \mathbb{E}_{(s_t, a_t) \sim \mathcal{D}} \left[\log \pi_{\theta}(a_t | s_t) \exp \left(\frac{1}{\alpha} \left(\sum_{t'=t}^T r(s_{t'}, a_{t'}) - \hat{V}_{\phi}^{\pi_{\beta}}(s_t) \right) \right) \right]$$

Symbol	What it means
$\hat{V}_{\phi}^{\pi_{\beta}}(s_t)$	The estimated value of state s_t for the behavior policy π_{β} that produced the data (not for π_{θ}).
$\sum_{t'=t}^T r(s_{t'}, a_{t'})$	The actual total reward observed from time t to the end of that trajectory (the Monte Carlo return).
α	A temperature hyperparameter that keeps the exponential weight from exploding.

What the equations are saying. The first equation fits $\hat{V}$ using ordinary regression toward the observed return — exactly like fitting any supervised model, with no learned bootstrap target involved. The second equation is weighted imitation (Slide 20), where the advantage is (observed return — estimated value), scaled by $1/\alpha$.

Understanding α . Small α makes the weighting sharper — close to only imitating the very best actions (similar to filtered BC). Large α flattens the weighting — close to imitating everything roughly equally, i.e., close to plain, unweighted BC.

Worked example: the deterministic-behavior-policy edge case. If the behavior policy π_{β} always takes exactly the same single action in a given state, then the observed return from that state already *equals* $\hat{V}^{\pi_{\beta}}(s)$ for every visit, so the advantage is 0 for every sample, every weight is $\exp(0) = 1$, and AWR degenerates to exactly plain, unweighted behavior cloning.

Aspects of this concept.

- **Advantage:** very simple value-fitting step (ordinary regression, no bootstrapping instability); never queries or trains on any out-of-distribution action, because it only ever uses returns actually observed following data actions.
- **Disadvantage 1:** Monte Carlo returns are noisy — a single trajectory’s full return is a high-variance sample of the true expected value.

- **Disadvantage 2:** $\hat{V}^{\pi_\beta}$ is an estimate for the (typically weaker) behavior policy π_β , not for the policy π_θ we are actually trying to improve — so the weighting is somewhat mismatched with the policy we hope to end up with.
- **The interpretation trap:** a high AWR weight on an action that appears only once or twice in the dataset should not be trusted — with so little data, the estimate of Q (via the observed return) for that action is itself high-variance, and exponentiating amplifies that noise rather than correcting it. AWR trusts its estimates blindly; it has no explicit notion of “how confident should I be here.”
- **When to use it:** when a simple, stable, OOD-safe offline algorithm is wanted and Monte Carlo’s variance and behavior-policy mismatch are acceptable.
- **What if this were done differently?** If $\hat{V}$ were instead fit with TD (bootstrapping) rather than Monte Carlo, and targeted at π_θ rather than π_β , we would arrive at AWAC (Slide 23) — lower variance and on-policy-relevant values, but at the cost of reopening the OOD-query problem.

20 Slide 22: Monte Carlo Estimation — A Trade-off Summary

What is the concept? A short pros-and-cons summary of using Monte Carlo returns (as AWR does) to estimate value.

+	Simple: it is ordinary supervised regression toward an observed number.
+	Completely avoids querying or training on any out-of-distribution action, since it only ever uses returns that were literally observed after real, data-recorded actions.
–	Monte Carlo estimation is noisy (high variance): a single trajectory’s return is only one sample of a random outcome.
–	$\hat{A}^{\pi_\beta}$ is an advantage estimate for the (weaker) behavior policy, not for the (better) target policy π_θ we actually want to end up with.

Aspects of this concept. This slide is the bridge between AWR and AWAC. The two weaknesses listed here — noisy estimates and being tied to the wrong (weaker) policy — are exactly what motivates replacing Monte Carlo with TD bootstrapping targeted at π_θ , which is the whole idea of the next slide (AWAC). The cost of doing so is reopening the out-of-distribution problem that Monte Carlo estimation structurally avoided (Slides 17–18) — this is a genuine trade-off, not a strict improvement.

21 Slide 23: Advantage-Weighted Actor-Critic (AWAC)

What is the concept? AWAC uses the exact same weighted-imitation recipe as AWR (Slide 20/21), but replaces the Monte Carlo value estimate with a TD (bootstrapped) estimate of Q^{π_θ} — the value *for the current policy*, not the old behavior policy.

Plain-English explanation. Instead of waiting to see a whole trajectory’s actual total reward (as AWR does), AWAC learns a Q-function the usual actor-critic way: bootstrap off of a one-step reward plus the estimated value of whatever the *current* policy would do next. This tends to be less noisy than a full Monte Carlo return, and (unlike AWR) it is actually an estimate *for the policy we are improving*, not the old data-collecting policy.

The equations.

$$\min_{\phi} \mathbb{E}_{(s,a,r,s') \sim \mathcal{D}} \left[\left(\hat{Q}_{\phi}^{\pi_{\theta}}(s, a) - \left(r + \gamma \mathbb{E}_{a' \sim \pi_{\theta}(\cdot | s')} [\hat{Q}_{\phi}^{\pi_{\theta}}(s', a')] \right) \right)^2 \right]$$

$$\theta \leftarrow \arg \max_{\theta} \mathbb{E}_{s,a \sim \mathcal{D}} \left[\log \pi_{\theta}(a | s) \exp(\hat{A}^{\pi_{\theta}}(s, a)) \right], \quad \hat{A}^{\pi_{\theta}}(s, a) = \hat{Q}_{\phi}^{\pi_{\theta}}(s, a) - \mathbb{E}_{\bar{a} \sim \pi_{\theta}(\cdot | s)} [\hat{Q}_{\phi}^{\pi_{\theta}}(s, \bar{a})]$$

Symbol	What it means
$\hat{Q}_{\phi}^{\pi_{\theta}}$	The critic, estimated via TD (one-step bootstrapping), for the current policy π_{θ} (not for the behavior policy π_{β} that produced the data).
$a' \sim \pi_{\theta}(\cdot s')$	A freshly sampled next action from the current policy — this is the out-of-distribution query, noted explicitly in red on the slide.
An alternative noted on the slide	$a' \sim \mathcal{D}$: sample the next action from the dataset instead, to avoid the OOD query (at a cost, discussed below).
$\bar{a} \sim \pi_{\theta}(\cdot s)$	A freshly sampled action used only to compute $V(s)$ as the average Q over the current policy’s own choices.

What the equations are saying. The critic equation is a standard actor-critic TD update, except the “next action” a' used in the bootstrap target comes from the *current* policy π_{θ} , not from the dataset. The policy equation is exactly weighted imitation learning (Slide 20), with the advantage now coming from this TD-based, current-policy critic instead of Monte Carlo.

Aspects of this concept.

- **Advantage:** because the critic is $Q^{\pi_{\theta}}$ (current policy) rather than $Q^{\pi_{\beta}}$ (behavior policy), and because TD estimates are typically lower-variance than Monte Carlo returns, AWAC’s advantage estimates are both more relevant and less noisy than AWR’s — allowing it to potentially improve beyond the behavior policy in a way AWR structurally cannot. It also still only ever trains the actor’s log-likelihood term on actions actually present in the dataset ($a \in \mathcal{D}$).
- **Disadvantage:** the critic’s bootstrap target samples $a' \sim \pi_{\theta}(\cdot | s')$ — exactly the out-of-distribution query move from Slides 17–18. AWAC therefore reopens the Q-overestimation problem that AWR completely avoided. In a fixed offline dataset with no way to check against reality, this is genuinely risky.
- **Trade-off table (the three offline algorithms so far):**

	Value estimate	Touches OOD actions?	Weakness
AWR	Monte Carlo $\hat{V}^{\pi_\beta}$	No	Monte Carlo is noisy; the advantage is for the weaker behavior policy, not π_θ .
AWAC	TD $\hat{Q}^{\pi_\theta}, a' \sim \pi_\theta$	Yes (in the critic)	Reintroduces OOD queries and their overestimation risk.

(IQL, the third row of this table, is completed in Slide 26.)

- **What if this were done differently?** The slide’s own noted alternative — sampling $a' \sim \mathcal{D}$ instead of $a' \sim \pi_\theta(\cdot | s')$ — removes the OOD query, but then the critic is again evaluating something closer to Q^{π_β} , undoing exactly the advantage AWAC was trying to gain over AWR. This is the same underlying trade-off already seen in Slides 17–18.

22 Slide 24: Expectile Regression — Motivation

What is the concept? This slide poses the question that motivates the next two slides: instead of estimating the *mean* of a random variable (which ordinary squared-error loss always gives you), can we estimate something closer to a high percentile — an “optimistic but still data-grounded” summary — while still only ever looking at actions that actually appear in the data?

Plain-English explanation. Picture, for a fixed state s' , a histogram of $Q(s', a)$ values for all the different actions a that the behavior policy actually tried there. Ordinary ℓ_2 (squared error) regression will always find you the *average height* of this histogram. But what we would really like, to avoid ever needing to query an action outside the data, is something like “the value of the *best* action that was actually tried” — closer to a high percentile of the histogram, computed *only* using the actions that are genuinely there.

Aspects of this concept.

- **Problem it is trying to solve:** get an estimate of $V(s)$ that behaves like “the best of what was tried” rather than “the average of what was tried,” without ever plugging in an action that was never tried (i.e., without any $\max_{a'} Q(s, a')$ over all conceivable actions).
- **Why ordinary ℓ_2 loss cannot do this:** squared-error loss is minimized exactly at the mean — it has no built-in way to lean toward the high or low end of a distribution.
- **What problem it gives rise to:** the need for an asymmetric loss function that can be pushed toward a higher or lower “expectile” — this is exactly the expectile loss defined on the next slide.

23 Slide 25: The Expectile Loss

What is the concept? A generalization of ordinary squared-error loss with one extra knob, λ , that lets the fitted value lean toward the upper or lower part of a distribution instead of always sitting exactly at the mean.

Plain-English explanation. Ordinary squared error treats being-too-high and being-too-low exactly the same way. The expectile loss instead makes one direction of error more expensive than the other, controlled by a single knob λ between 0 and 1. Whichever direction is made more *expensive*, the best-fit value will shift *away* from making that kind of error — which pulls the fit toward the *opposite* side of the distribution.

The equation.

$$\ell_2^\lambda(x) = \begin{cases} (1 - \lambda) x^2 & \text{if } x < 0 \\ \lambda x^2 & \text{otherwise} \end{cases}$$

Symbol	What it means
x	The residual (error): what is being subtracted from what depends on the specific application (defined explicitly for IQL on the next slide).
λ	The expectile level, a number between 0 and 1 that decides which direction of error is penalized more.
$\ell_2^\lambda(x)$	The loss value: an ordinary squared error, but scaled differently depending on the sign of x .

What the equation is saying. If x is negative, the error is scaled by $(1 - \lambda)$; if x is zero or positive, it is scaled by λ . At $\lambda = 0.5$, both sides are scaled equally and this is exactly ordinary squared error (which gives back the mean — $\lambda = 0.5$ is the “50th expectile,” analogous to the median). As λ moves toward 1, positive x becomes very expensive, so the best fit shifts *down* to avoid producing positive residuals as often (a lower expectile). As λ moves toward 0, negative x becomes very expensive, so the best fit shifts *up* (a higher, more optimistic expectile).

Aspects of this concept.

- **What it is:** an asymmetric squared-error loss; a strict generalization of ordinary mean-squared error, which is the special case $\lambda = 0.5$.
- **Advantage:** lets us target an “expectile” (a percentile-like statistic) instead of only the mean, while remaining smooth and easy to optimize with gradients — unlike quantile regression, which uses an absolute-value loss that is not smooth at zero.
- **Disadvantage:** an expectile is a related but technically different statistic from a literal percentile (it is defined by this asymmetric-loss minimization, not by sorting the data), so the intuition “ $\lambda = 0.9$ means the 90th percentile” is only approximate; it also introduces a new hyperparameter, λ , that must be chosen.
- **Problem it solves:** gives a tunable knob for whether a fitted value leans optimistic (upper expectile) or pessimistic (lower expectile).
- **Problem it does not solve by itself:** this slide does not say what x actually represents in a specific algorithm — that choice is made on the very next slide, for IQL.
- **When to use it:** whenever you want a value estimate that reflects near-best (or near-worst) outcomes rather than the average, using only the data you actually have.

- **What if this were done differently?** Using an absolute-value (quantile) loss instead of a squared-error-based one would give literal quantiles rather than expectiles, at the cost of losing the smooth, easy-to-optimize gradient that squared error provides.

24 Slide 26: Implicit Q-Learning (IQL)

What is the concept? A full, three-step offline RL algorithm that uses expectile regression to estimate an optimistic value function *without ever evaluating Q on an action outside the dataset* — completing the trade-off table started in Slide 23.

Note on the slide. Compare this slide’s formula for V , which uses $x = V(s) - \hat{Q}(s, a)$ and instructs “using small $\lambda < 0.5$,” against the expectile loss on Slide 25, which used $x < 0$ to mean the $(1 - \lambda)$ -weighted branch. Because $x = V - Q$ here, $x < 0$ means V is *below* Q (i.e., V underestimates Q). With small λ , that branch is weighted by $(1 - \lambda)$, which is close to 1 — i.e., underestimating Q is made *expensive*. So the best-fit V shifts *up*, toward the upper expectile, exactly as intended. This is self-consistent with the slide’s own instruction. It is worth knowing, however, that this labeling is the mirror image of how the original IQL paper writes the same loss (using $x = Q - V$ and a λ close to 1 to mean the upper expectile) — both conventions produce the identical intended behavior (an optimistic V), they simply place the label “small λ ” versus “large λ ” on opposite sides. On the exam, state which convention you are using in one sentence, then answer using the mechanism (“ V shifts above the mean”), not just the raw label.

Plain-English explanation of the full pipeline.

1. Fit $\hat{V}(s)$ to be an optimistic-but-honest summary of $\hat{Q}$ at each state, using expectile regression, restricted entirely to the (s, a) pairs that are actually in the dataset. This behaves like a soft “best of what was tried” summary.
2. Fit $\hat{Q}(s, a)$ with an ordinary squared-error TD update, using $\hat{V}(s')$ — not a max or expectation over next actions — as the bootstrap target. This is the crucial trick: it never needs to evaluate Q at any next action at all.
3. Extract an actual usable policy using the same weighted-imitation recipe as AWR and AWAC.

The equations.

$$\hat{V}(s) \leftarrow \arg \min_V \mathbb{E}_{(s,a) \sim \mathcal{D}} \left[\ell_2^\lambda(V(s) - \hat{Q}(s, a)) \right] \quad (\text{using small } \lambda < 0.5)$$

$$\hat{Q}(s, a) \leftarrow \arg \min_Q \mathbb{E}_{(s,a,s') \sim \mathcal{D}} \left[\left(Q(s, a) - (r + \gamma \hat{V}(s')) \right)^2 \right]$$

$$\hat{\pi} \leftarrow \arg \max_{\pi} \mathbb{E}_{s,a \sim \mathcal{D}} \left[\log \pi(a | s) \exp \left(\frac{1}{\alpha} (\hat{Q}(s, a) - \hat{V}(s)) \right) \right]$$

Symbol	What it means
$\hat{V}(s)$	The optimistic value estimate, fit via expectile regression using only dataset (s, a) pairs.
$\hat{Q}(s, a)$	The Q-function, fit with plain TD (squared-error) regression, bootstrapped off $\hat{V}(s')$ instead of any action-dependent next-step quantity.
λ	The expectile level for the V -step; small (as instructed) pushes $\hat{V}$ toward the upper expectile of $\hat{Q}$ over dataset actions.
α	The temperature for the final weighted-imitation policy-extraction step (same role as in AWR/AWAC).

Why IQL never touches OOD actions (the key justify-the-design point). Standard Q-learning’s bootstrap target needs $\max_{a'} Q(s', a')$ or $\mathbb{E}_{a' \sim \pi_\theta}[Q(s', a')]$ — both of which require evaluating Q at actions that might never appear in the dataset. IQL replaces that entire max/expectation with $\hat{V}(s')$, and $\hat{V}$ is fit using an expectile loss computed *only* over the (s, a) pairs that are actually present in $\mathcal{D}$. The “soft max-like,” optimistic behavior comes entirely from re-weighting *existing* data through the asymmetric loss, never from plugging in an unseen action. This is why IQL avoids out-of-distribution queries *by construction*, not by luck.

Aspects of this concept.

- **Advantage:** gets the low-variance, current-policy-relevant benefits of TD bootstrapping (like AWAC) while completely avoiding the OOD-query problem that AWAC reintroduces.
- **Disadvantage:** introduces an extra hyperparameter, λ (the expectile level), which must be tuned; pushed too aggressively toward the extreme expectile, it can become overconfident about state-action pairs that are only sparsely represented in the data, since expectile regression from very few noisy samples is itself imprecise.
- **Completed trade-off table:**

	Value estimate	Touches OOD?	Weakness
AWR	Monte Carlo $\hat{V}^{\pi_\beta}$	No	Noisy; advantage is for the weaker behavior policy, not π_θ .
AWAC	TD $\hat{Q}^{\pi_\theta}, a' \sim \pi_\theta$	Yes (in the critic)	Reintroduces OOD queries.
IQL	TD with expectile $\hat{V}$	No	Extra hyperparameter λ ; conservative, can be imprecise on sparsely-represented actions.

- **When to use it:** offline settings where avoiding OOD overestimation matters most, and TD’s efficiency advantage over Monte Carlo is desired.
- **When not to use it:** very simple datasets from an essentially deterministic behavior policy, where plain BC or AWR is already sufficient and IQL’s extra machinery adds complexity without benefit.

- **What if this were done differently?** If the V -step used plain ℓ_2 regression ($\lambda = 0.5$) instead of an asymmetric expectile loss, $\hat{V}$ would just be the ordinary *mean* of $\hat{Q}$ over the data’s actions at that state — exactly the plain, non-optimistic estimate that this whole section is trying to move beyond.

25 Quick Reference: The Progression of Ideas

Starting point	Problem discovered	Fix introduced
Behavior cloning (ℓ_2)	Mean-collapse on multi-modal experts	Maximum likelihood with expressive distributions (GMM, autoregressive, diffusion/flow)
Expressive behavior cloning	Compounding error / distribution shift	DAgger (interactive data aggregation)
DAgger	Needs an interactive expert, often infeasible	Offline RL: reuse existing reward-labeled data instead
Offline data, naive off-policy actor-critic	Out-of-distribution Q-value overestimation	Restrict training to actions in the data: filtered BC, then weighted imitation
Filtered BC	Whole-trajectory filtering is too coarse	Weighted imitation with a per- (s, a) advantage (AWR)
AWR (Monte Carlo)	Noisy; advantage is for the weak behavior policy	AWAC (TD, current-policy advantage)
AWAC (TD)	Reopens OOD queries in the critic	IQL (TD with expectile $\hat{V}$, no OOD queries)

26 Key Errors and Ambiguities in the Source Slides (Review Before the Exam)

1. **Slide 11, “Diffusion Policy”:** the procedure shown (linear interpolation between noise and data, a single learned velocity field, Euler-integrated inference) is technically *flow matching* / *rectified flow*, not classic diffusion (DDPM), which uses many small noising steps and a stochastic reverse process. Answer exam questions using the slide’s own procedure; optionally note the more precise name for extra precision.
2. **Slides 25–26, expectile λ convention:** Slide 26 defines the residual as $x = V(s) - \hat{Q}(s, a)$ and instructs “using small $\lambda < 0.5$ ” to push $\hat{V}$ toward the upper expectile (this is internally self-consistent and verified numerically above). This is the mirror image of the original IQL paper’s own convention, which defines the residual the other way around ($x = Q - V$) and uses λ close to 1 for the same upper-expectile behavior. State your convention in one sentence before answering any question that asks about the direction of λ .